

Dimensionality Reduction Approaches to Insider Threat Detection: From Principal Components to Interpretable Sparse Features

by

Suramya Raj Angdembay

A Thesis

Submitted to the Honors College of
The University of Southern Mississippi
in Partial Fulfillment
of Honors Requirements

Month Year

Approved by:

Dr. Haiyan Tian, Degree

School of Mathematics and Natural Sciences

Thesis Co-Advisor Name, Degree

School of Your School

Director Name, Degree

School of Your School

Joyce Inman, Ph.D., Dean

Honors College

ABSTRACT

Insider threat represents a major cybersecurity challenge to companies, organizations, and government agencies. Detecting malicious insiders is difficult due to massive data volumes, extreme class imbalance, and the subtle nature of behavioral shifts over time. This thesis approaches the problem through dimensionality reduction: describing high-dimensional behavior with a small number of informative directions, and treating what a model of normal behavior fails to explain as a sign of anomaly. First, a classical approach uses Principal Component Analysis (PCA) to reduce behavioral features to their dominant directions of variance, reconstructs each observation from those directions with the inverse PCA transformation, and scores it by its reconstruction error. Second, a large language model is fine-tuned only on benign user-days written as text, with the fine-tuning restricted to low-rank changes of the model’s weights, and it scores each day by its average surprisal, the negative log-probability of each token given the tokens before it. We show that this score splits exactly into a profile part, covering organizational and personality attributes, and a behavior part, and that the profile part ranks users backwards: on CERT release 4.2, profile tokens alone place malicious users below benign ones (user-level AUC 0.47), while scoring behavior tokens alone raises user-level AUC from 0.65 to 0.86. Third, we apply an interpretable form of dimensionality reduction to the model itself. A sparse autoencoder, an overcomplete and sparse relative of PCA, is learned on the change that fine-tuning made to the model’s internal representations. Four of the five features most associated with malicious days activate almost only on behavioral tokens. On malicious users held out from feature selection, the best of many searched edits that move these features toward a benign colleague’s values, rather

than another malicious user’s, lowers the anomaly score more than the same comparison for control features, by a small margin; averaged over colleagues instead of taking the best, the sign reverses. Splitting the same edits by token class shows that, at full strength, they raise the loss on behavior tokens more than control edits do, while their effect on profile tokens is uncertain. This is evidence about a joint edit of the selected features. Where a feature activates, what it represents and which predictions its edit changes are separate questions. All experiments use the CMU CERT insider threat dataset.

Keywords: Insider Threat; Dimensionality Reduction; Principal Component Analysis (PCA); Low-Rank Adaptation; Surprisal; Large Language Models; Mechanistic Interpretability; Sparse Autoencoders; One-Class Anomaly Detection

DEDICATION

Your dedication text here.

ACKNOWLEDGMENTS

Your acknowledgments begin here. Recognize advisors, faculty, funding agencies, and institutional support.

Contents

ABSTRACT	2
DEDICATION	4
ACKNOWLEDGMENTS	5
LIST OF ABBREVIATIONS	10
1 Introduction	11
1.1 Example 1st Level Heading	11
Example 2nd Level Heading	11
2 Principal Component Analysis	12
2.1 Overview and Motivation	12
2.2 PCA in the p -Dimensional Setting	12
2.3 Inverse PCA and Reconstruction Error	14
2.4 A Numerical Example of PCA and Reconstruction	15
2.5 Low-Dimensional Geometric Intuition	15
2.6 A High-Dimensional Example: A 17-Dimensional Dataset	16
2.7 Synthetic anomaly example	16
2.8 Summary	20
3 Experimental Setup and Datasets	25
3.1 The CMU CERT Insider Threat Dataset	25

3.2	The Insider Case	26
3.3	Data Preparation for PCA Reconstruction Analysis	28
3.4	Data Preparation for Language-Model Analysis	29
3.5	Experimental Design	31
4	Language-Model Surprisal and Its Decomposition	33
4.1	Overview and Motivation	33
4.2	Writing a User-Day as Text	33
4.3	Next-Token Probability and Surprisal	35
4.4	Why the Negative Logarithm Measures Surprise	35
4.5	Learning From Benign Days Only	36
4.6	The Anomaly Score of a Day and of a User	37
4.7	Splitting the Score by Token Class	38
4.8	A Numerical Example	38
4.9	Measuring Detection: The User-Level AUC	39
4.10	Core Finding	40
5	Interpretable Dimensionality Reduction of the Fine-Tuned Model	42
5.1	Overview and Motivation	42
5.2	Hidden States and the Adaptation Difference	42
5.3	A Sparse Dictionary for the Difference	43
5.4	A Numerical Example of Sparse Coding	44
5.5	Choosing the Features to Test	44
5.6	Where Features Fire: Token-Class Attribution	45
5.7	Causal Test: Editing Features and Re-Scoring	45
5.8	Core Mechanistic Finding	47
5.9	Limitations	50
	REFERENCES	51

List of Tables

2.1	Rank-1 reconstruction of the four observations.	15
3.1	The three insider scenarios of CERT release 4.2, following the answer key. . .	26
3.2	User-disjoint splits of CERT release 4.2.	30
3.3	The three experiments.	31
4.1	Class means and scores of the two example days.	39
4.2	User-level AUC on CERT release 4.2 (60 malicious users, 79 benign users, none seen in training) for scores built from different token classes. Intervals are 95% bootstrap intervals, resampling malicious users.	40
5.1	PCA and the sparse autoencoder side by side.	44
5.2	Share of each selected feature’s activation on each token class, over malicious days of CERT release 4.2 (layer 26). Session (SES) lines are 74.9% of these tokens, the organization (DAY) line 12.6%, and the personality (PSY) line 9.0%. 47	47

List of Figures

2.4	Synthetic two-dimensional dataset with principal component directions drawn from the center of the data.	17
2.5	Dataset represented in the PCA coordinate system where each observation is described by its PC1 and PC2 coordinates.	18
2.6	Values of the first principal component for all observations.	19
2.7	Values of the second principal component for all observations.	20
2.1	Illustration of PCA in two dimensions: PCA rotates the coordinate system so that PC1 captures the largest variance while PC2 captures the smallest. Projecting onto PC1 yields a one-dimensional representation with minimal loss when PC2 contributes little variance.	22
2.2	Illustration of PCA in three dimensions: PC1, PC2, and PC3 form an orthonormal basis with decreasing explained variance. Dropping PC3 projects data onto a best-fit plane.	23
2.3	A high-dimensional PCA illustration (17 features): PCA reveals dominant axes of variation among observations (countries). Large separation along PC1 indicates an observation that differs substantially along the primary pattern in the data.	24

LIST OF ABBREVIATIONS

APA	American Psychological Association
AUC	Area Under the (Receiver Operating Characteristic) Curve
CERT	CERT Division of the Software Engineering Institute
LoRA	Low-Rank Adaptation
PCA	Principal Component Analysis
QLoRA	Quantized Low-Rank Adaptation
ROC	Receiver Operating Characteristic
SAE	Sparse Autoencoder
USM	The University of Southern Mississippi

Chapter I

Introduction

Begin the first chapter here. Indent the first line of each paragraph.

1.1 Example 1st Level Heading

Text for first-level heading.

Example 2nd Level Heading

Text for second-level heading.

Example 3rd Level Heading (In Paragraph)

Example 3rd Level Heading. Continue paragraph text immediately after the bold heading.

Chapter II

Principal Component Analysis

2.1 Overview and Motivation

Principal Component Analysis (PCA) is a classical technique for identifying orthogonal directions along which the data exhibits the greatest variance. In applications such as behavioral cybersecurity logs, the raw feature space can be high-dimensional and strongly correlated, which complicates visualization, statistical modeling, and anomaly detection. PCA addresses these issues by constructing an orthonormal coordinate system whose axes (the *principal components*) are ordered by the amount of variance they explain. This chapter presents the mathematical formulation of PCA in the general p -dimensional setting, the inverse transformation that reconstructs data from the reduced space, and the reconstruction error that motivates PCA-based anomaly scoring, followed by low- and high-dimensional illustrations. Historically, the geometric origins of PCA trace back to Pearson, while the familiar algebraic eigenvalue formulation was developed by Hotelling [1, 2, 3].

2.2 PCA in the p -Dimensional Setting

PCA assumes that each observation is a numerical vector in $\mathbb{R}^p$, that the structure of the data can be approximated by a linear subspace, that directions of larger variance carry more meaningful variation, and that Euclidean distance is meaningful in the feature space. Because PCA is sensitive to the relative magnitude of features, the features are standardized

first.

Let $X \in \mathbb{R}^{n \times p}$ be a data matrix with n observations and p features, whose i th row is the observation $\mathbf{x}_i^\top$. With the sample mean $\bar{\mathbf{x}} = \frac{1}{n} \sum_{i=1}^n \mathbf{x}_i$ and the vector of ones $\mathbf{1} \in \mathbb{R}^n$, the centered data matrix is

$$X_c = X - \mathbf{1}\bar{\mathbf{x}}^\top,$$

which subtracts the mean from every observation. The sample covariance matrix is

$$S = \frac{1}{n-1} X_c^\top X_c \in \mathbb{R}^{p \times p}, \quad (2.1)$$

which is symmetric and positive semidefinite and therefore has an orthonormal eigendecomposition.

PCA looks for the unit direction $\mathbf{a}$ along which the projected scores $z_i = \mathbf{a}^\top(\mathbf{x}_i - \bar{\mathbf{x}})$ vary most. Their variance is $\mathbf{a}^\top S \mathbf{a}$, so the first principal direction solves

$$\max_{\mathbf{a} \in \mathbb{R}^p} \mathbf{a}^\top S \mathbf{a} \quad \text{subject to} \quad \mathbf{a}^\top \mathbf{a} = 1. \quad (2.2)$$

With a Lagrange multiplier λ , setting the gradient of $\mathbf{a}^\top S \mathbf{a} - \lambda(\mathbf{a}^\top \mathbf{a} - 1)$ to zero gives

$$S \mathbf{a} = \lambda \mathbf{a}. \quad (2.3)$$

The principal directions are therefore eigenvectors of S , and each eigenvalue is the variance explained along its direction [2, 3]. Ordering the eigenvalues $\lambda_1 \geq \dots \geq \lambda_p \geq 0$ with orthonormal eigenvectors $\mathbf{u}_1, \dots, \mathbf{u}_p$ gives $S = U \Lambda U^\top$, with $U = [\mathbf{u}_1 \ \dots \ \mathbf{u}_p]$ and $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_p)$.

To reduce the dimension to $q < p$, keep the first q directions, $U_q = [\mathbf{u}_1 \ \dots \ \mathbf{u}_q] \in \mathbb{R}^{p \times q}$. The reduced data, or *principal component scores*, are

$$Z = X_c U_q \in \mathbb{R}^{n \times q}, \quad (2.4)$$

whose i th row is $\mathbf{z}_i^\top = (U_q^\top \mathbf{x}_{c,i})^\top$. Since the total variance is $\text{tr}(S) = \sum_{j=1}^p \lambda_j$, the proportion of variance explained by the first q components is

$$\text{PVE}(q) = \frac{\sum_{j=1}^q \lambda_j}{\sum_{j=1}^p \lambda_j}, \quad (2.5)$$

and q is commonly chosen so that $\text{PVE}(q)$ exceeds a fixed threshold such as 0.95.

2.3 Inverse PCA and Reconstruction Error

PCA provides dimensionality reduction, and the *inverse PCA transformation* approximates the original data from the reduced space:

$$\hat{X} = Z U_q^\top + \mathbf{1} \bar{\mathbf{x}}^\top, \quad \hat{\mathbf{x}}_i = U_q \mathbf{z}_i + \bar{\mathbf{x}}, \quad (2.6)$$

where Z is the reduced data, U_q holds the principal components and $\bar{\mathbf{x}}$ is the mean vector. For the centered observation $\mathbf{x}_{c,i} = \mathbf{x}_i - \bar{\mathbf{x}}$ this is $\hat{\mathbf{x}}_{c,i} = U_q U_q^\top \mathbf{x}_{c,i}$, the orthogonal projection onto the principal subspace $\text{span}\{\mathbf{u}_1, \dots, \mathbf{u}_q\}$.

The reconstruction error measures the difference between the original and the reconstructed data. For one observation it is

$$e_i = \|\mathbf{x}_i - \hat{\mathbf{x}}_i\|_2^2 = \|(I - U_q U_q^\top) \mathbf{x}_{c,i}\|_2^2, \quad (2.7)$$

the squared Euclidean distance from $\mathbf{x}_{c,i}$ to the principal subspace, and over n observations it is the mean squared error

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n \|\mathbf{x}_i - \hat{\mathbf{x}}_i\|_2^2. \quad (2.8)$$

A model built from normal data reconstructs normal observations well, so a large reconstruction error may indicate an anomaly. This is the basis of the PCA anomaly score used in this thesis.

2.4 A Numerical Example of PCA and Reconstruction

Consider four observations of two features,

$$X = \begin{bmatrix} 2 & 1 \\ 3 & 2 \\ 4 & 2 \\ 5 & 4 \end{bmatrix}, \quad \bar{\mathbf{x}} = \begin{bmatrix} 3.5 \\ 2.25 \end{bmatrix}, \quad S = \begin{bmatrix} 1.6667 & 1.5 \\ 1.5 & 1.5833 \end{bmatrix}.$$

The eigenvalues of S are $\lambda_1 \approx 3.125$ and $\lambda_2 \approx 0.125$, so the first principal component $\mathbf{u}_1 \approx (0.72, 0.69)$ explains about 96% of the variance. Keeping $q = 1$, the scores are $z \approx (-1.94, -0.53, 0.19, 2.28)$, and the inverse transformation $\hat{\mathbf{x}}_i = z_i \mathbf{u}_1 + \bar{\mathbf{x}}$ gives the reconstructions and errors in Table 2.1.

Table 2.1: Rank-1 reconstruction of the four observations.

i	$\mathbf{x}_i$	$\hat{\mathbf{x}}_i$	$e_i = \ \mathbf{x}_i - \hat{\mathbf{x}}_i\ _2^2$
1	(2, 1)	(2.10, 0.91)	0.018
2	(3, 2)	(3.12, 1.88)	0.029
3	(4, 2)	(3.64, 2.38)	0.274
4	(5, 4)	(5.15, 3.82)	0.054

The third observation lies farthest from the line spanned by $\mathbf{u}_1$ and has the largest error, and $\text{MSE} = (0.018 + 0.029 + 0.274 + 0.054)/4 \approx 0.094$. The same operations apply unchanged in high dimensions.

2.5 Low-Dimensional Geometric Intuition

In $\mathbb{R}^2$, PCA identifies a new pair of orthonormal axes (u_1, u_2) obtained by rotating the original coordinate system so that u_1 aligns with the direction of maximum spread. If the

data are approximately distributed along a line, reducing from 2D to 1D projects the points onto the line spanned by u_1 and discards the orthogonal component (Figure 2.1). In $\mathbb{R}^3$, the three directions (u_1, u_2, u_3) are ordered by $(\lambda_1, \lambda_2, \lambda_3)$, and reducing to 2D projects onto the plane spanned by (u_1, u_2) (Figure 2.2).

2.6 A High-Dimensional Example: A 17-Dimensional Dataset

PCA is often most valuable when p is large and direct visualization is not possible. To illustrate this, consider a dataset where each observation corresponds to a country and each feature corresponds to a measured quantity, producing points in $\mathbb{R}^{17}$. For example, one may treat the weekly per-person consumption of 17 food categories as a vector in $\mathbb{R}^{17}$. Although we cannot visualize $\mathbb{R}^{17}$ directly, PCA provides low-dimensional views (e.g., $q = 1$ or $q = 2$) that reveal dominant contrasts among countries.

Let $x_k \in \mathbb{R}^{17}$ denote the centered consumption vector for country k . The first PC score is $z_{k1} = u_1^\top x_k$, and the second is $z_{k2} = u_2^\top x_k$. Countries separated strongly along PC1 exhibit the greatest differences along the dominant consumption pattern, while PC2 captures the second most important pattern not explained by PC1.

2.7 Synthetic anomaly example

To visualize how PCA organizes behavioral observations and highlights deviations from dominant patterns, we construct 30 two-dimensional points: 27 typical observations concentrated around a common trend, and three placed away from it to represent anomalous behavior. Figure 2.4 shows the data with the two principal directions drawn from its center; the first component aligns with the main trend and the second captures variation orthogonal to it. Projecting every observation onto these components gives the representation in Figure 2.5,

and Figures 2.6 and 2.7 plot each observation's coordinate along the first and second components. Points that deviate from the general pattern become more clearly distinguishable in their component coordinates.



Figure 2.4: Synthetic two-dimensional dataset with principal component directions drawn from the center of the data.

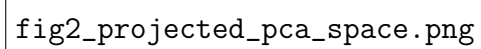


fig2_projected_pca_space.png

Figure 2.5: Dataset represented in the PCA coordinate system where each observation is described by its PC1 and PC2 coordinates.

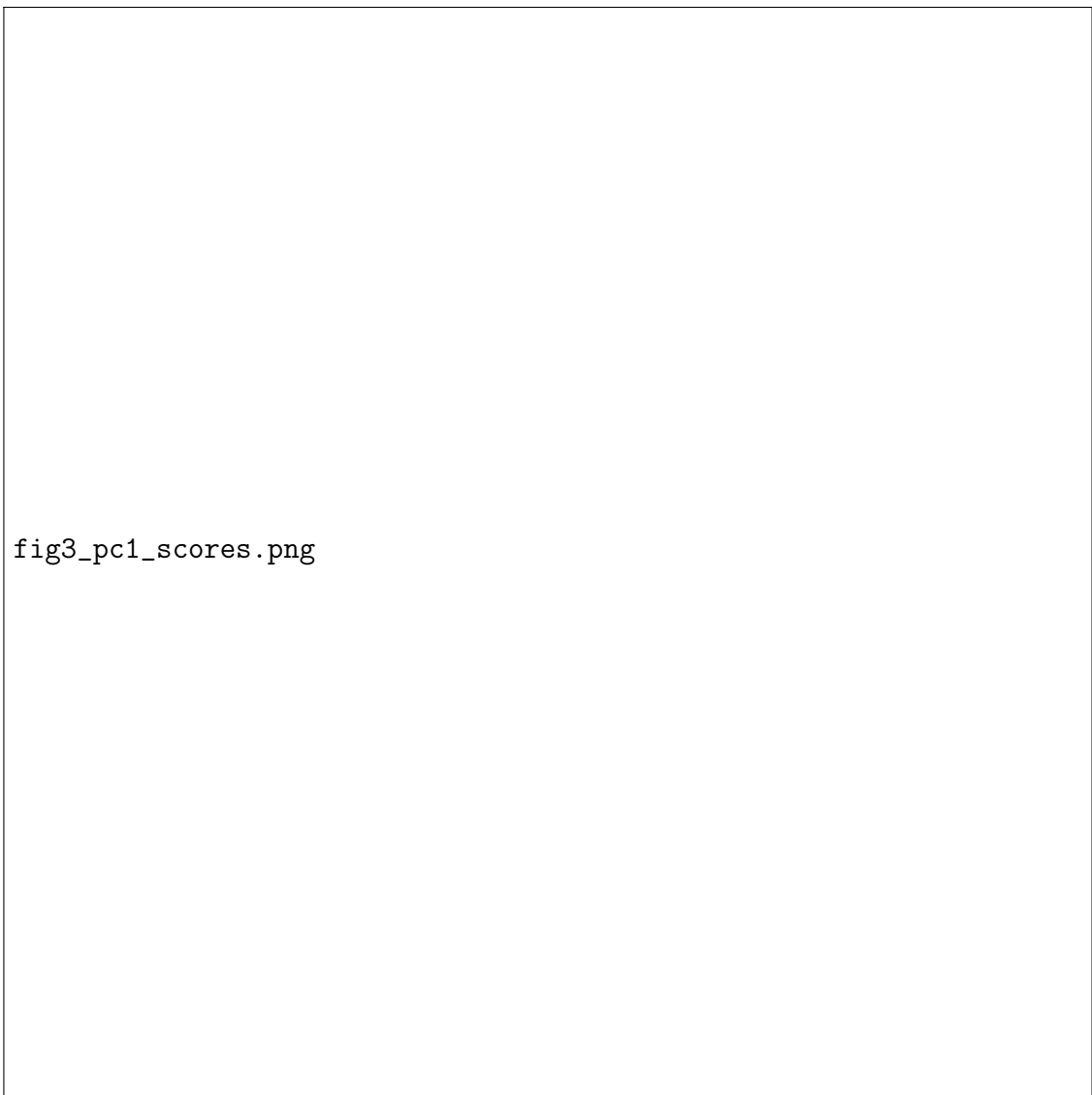


fig3_pc1_scores.png

Figure 2.6: Values of the first principal component for all observations.

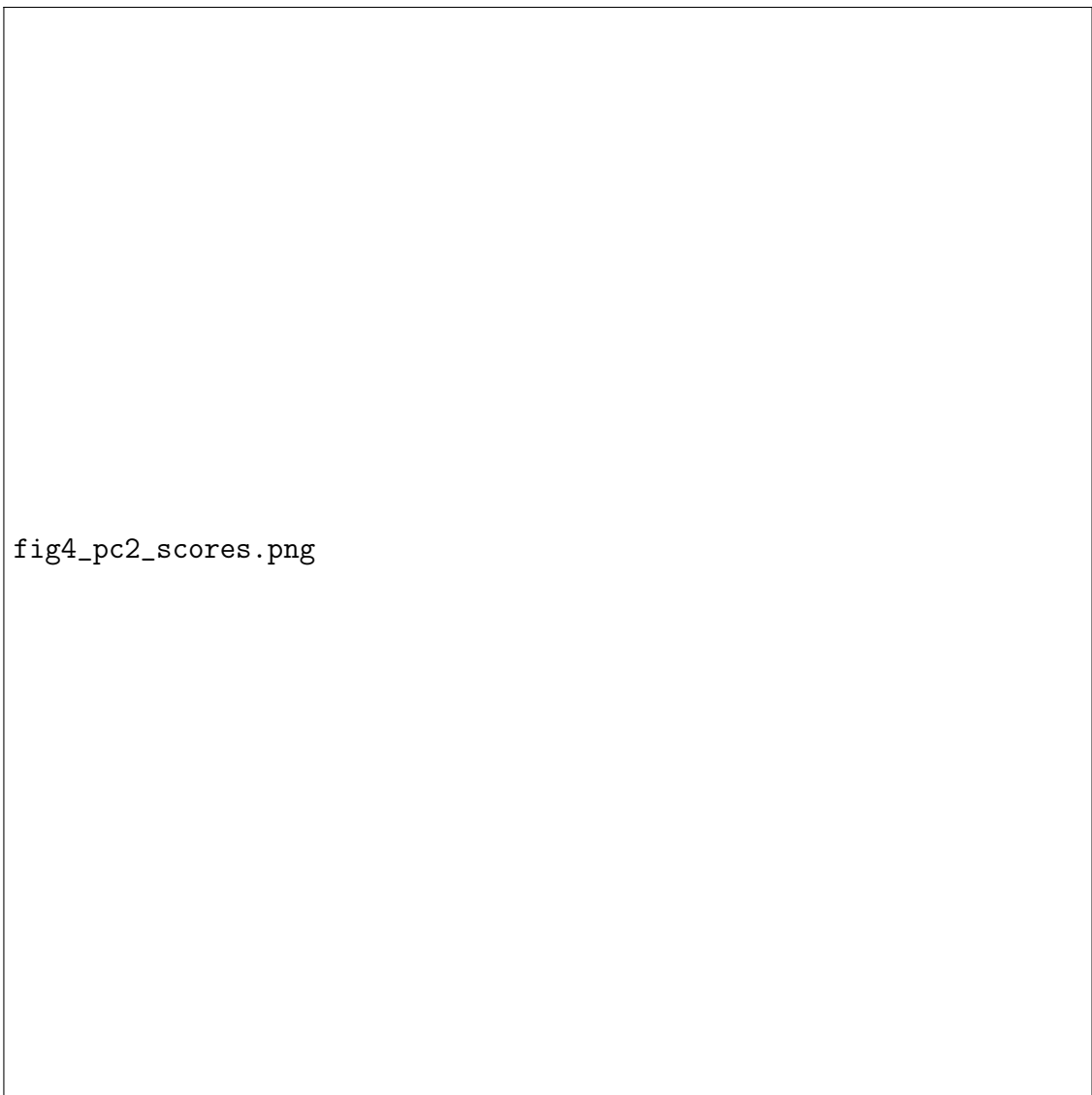


fig4_pc2_scores.png

Figure 2.7: Values of the second principal component for all observations.

2.8 Summary

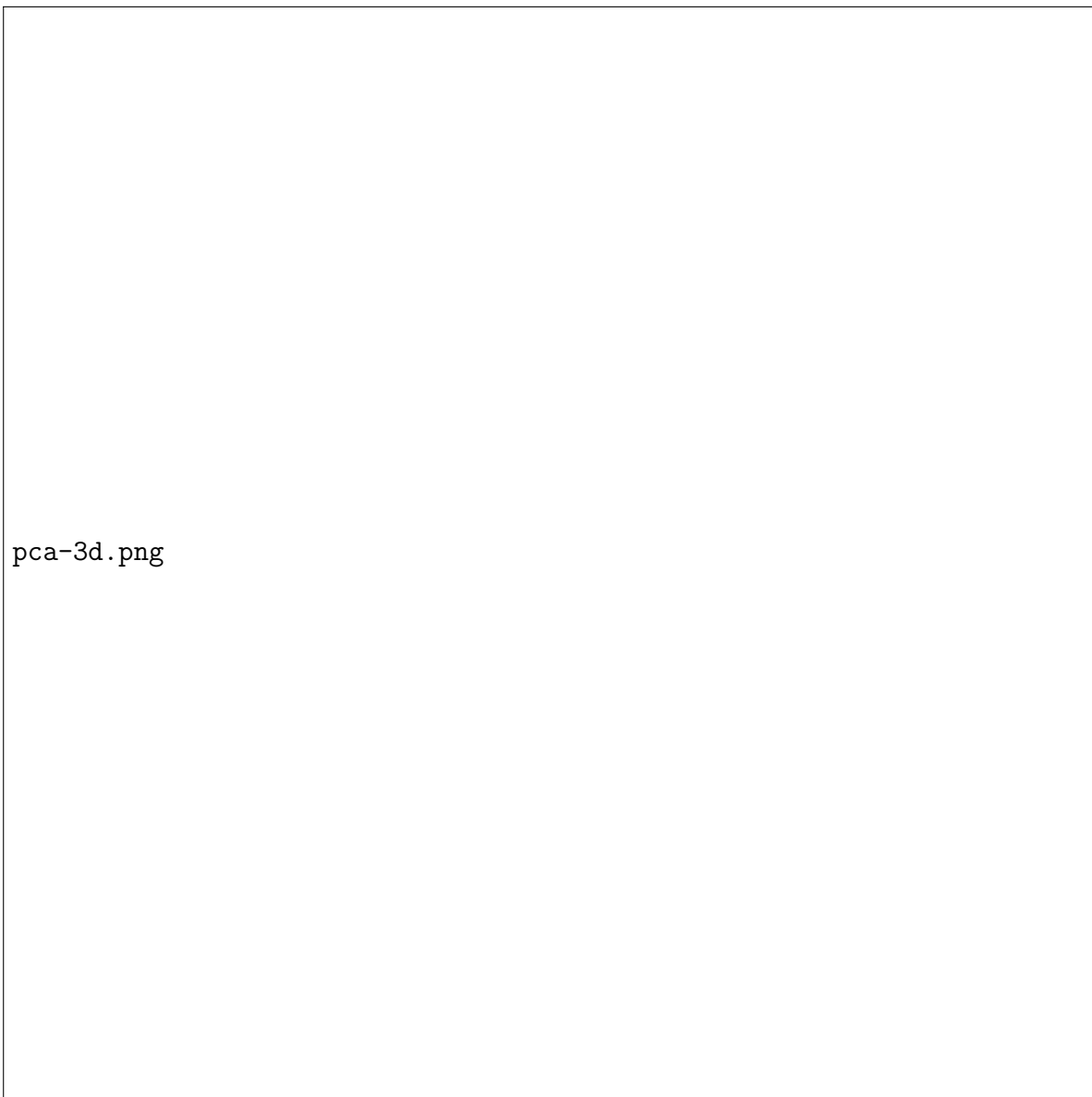
This chapter presented PCA in the general p -dimensional setting through the covariance matrix, its eigendecomposition and the variance-maximization principle, and interpreted it as an orthogonal projection onto a low-dimensional subspace. The inverse PCA transformation maps the reduced data back to the feature space, and the reconstruction error, and its mean over many observations, measures what the dominant directions fail to explain. That error

is the PCA anomaly score used in this thesis.



pca-2d.png

Figure 2.1: Illustration of PCA in two dimensions: PCA rotates the coordinate system so that PC1 captures the largest variance while PC2 captures the smallest. Projecting onto PC1 yields a one-dimensional representation with minimal loss when PC2 contributes little variance.



pca-3d.png

Figure 2.2: Illustration of PCA in three dimensions: PC1, PC2, and PC3 form an orthonormal basis with decreasing explained variance. Dropping PC3 projects data onto a best-fit plane.

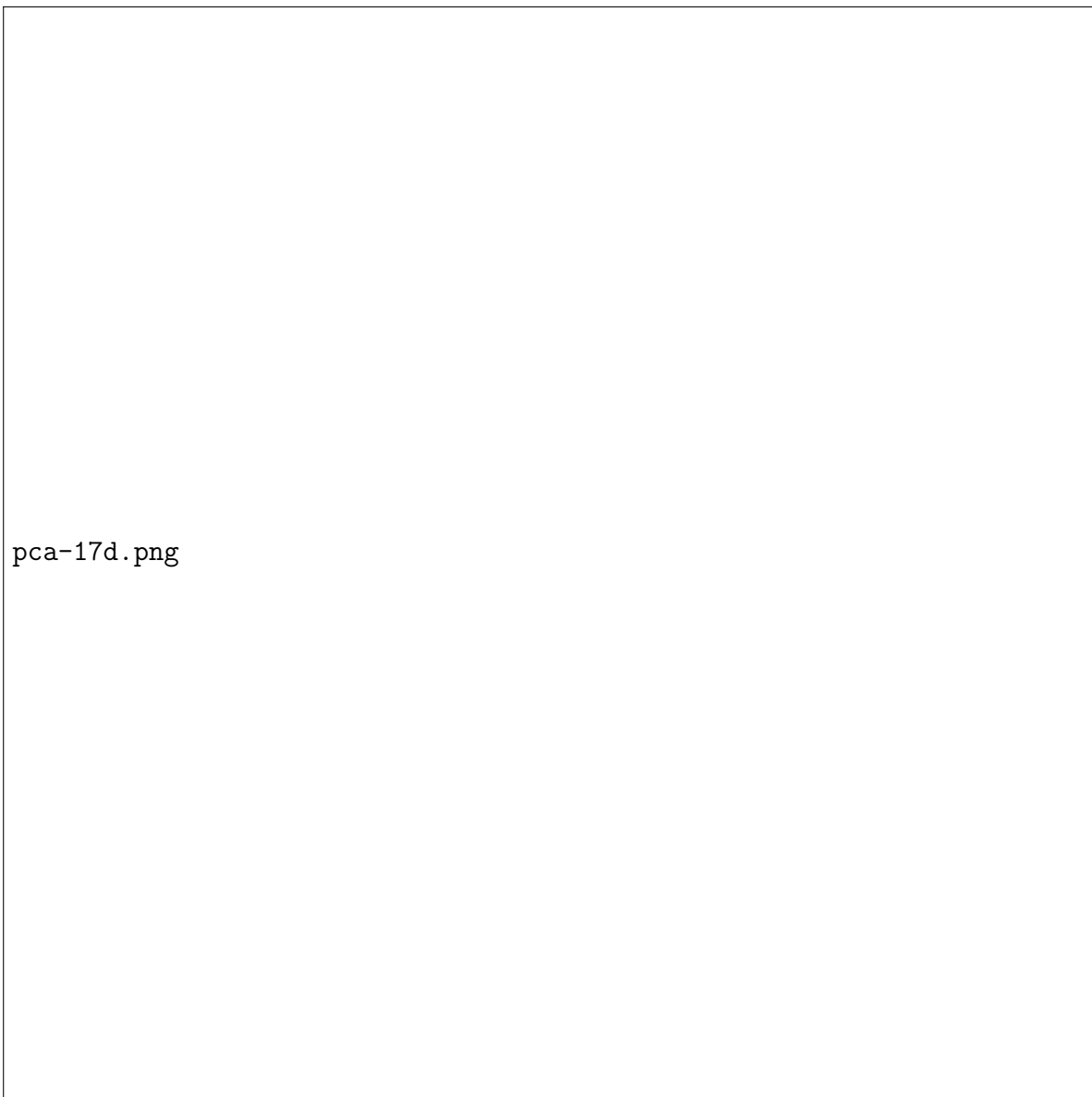


Figure 2.3: A high-dimensional PCA illustration (17 features): PCA reveals dominant axes of variation among observations (countries). Large separation along PC1 indicates an observation that differs substantially along the primary pattern in the data.

Chapter III

Experimental Setup and Datasets

Chapter II described a method that needs a data matrix X . Chapter IV needs a text for each day, and Chapter V needs the internal states of a model that has read those texts. All three study the same people, the same days and the same definition of a malicious act. This chapter fixes those shared objects: the dataset (Section 3.1), what counts as insider activity (Section 3.2), the two representations of the data (Sections 3.3 and 3.4), and the rules every experiment follows (Section 3.5).

3.1 The CMU CERT Insider Threat Dataset

The CERT Insider Threat Test Dataset is a collection of synthetic datasets produced by the CERT Division of Carnegie Mellon University’s Software Engineering Institute, with Exact-Data, LLC, under sponsorship from DARPA [5, 4]. Each dataset simulates an organization in which ordinary employees go about their work and a few simulated insiders act out a scripted malicious scenario. This thesis uses release 4.2, which simulates 1,000 employees whose recorded activity runs from 2 January 2010 to 17 May 2011. The data contain activity logs (logons and logoffs, removable-device connections, file operations, emails and web visits), organizational records (role, business unit, functional unit, department, team, and whether the employee is an IT administrator), and five personality scores for every employee.

Every log entry is an event $e = (u, \tau, s, \mathbf{a})$: a user $u \in \mathcal{U}$, a time τ , a source s (logon, device, file, email or web) and a list of attributes $\mathbf{a}$, with $|\mathcal{U}| = 1,000$. The unit of observation

is the *user-day* $D = (u, t)$, one user on one calendar day, whose events are $E_{u,t} = \{e : e \text{ is an event of } u \text{ on day } t\}$. Every method in this thesis maps a user-day to an anomaly score $s(D) \in \mathbb{R}$, and a larger score means a more unusual day.

3.2 The Insider Case

The dataset’s answer key names each simulated insider, the scenario that insider follows, and the malicious events involved [4]. Release 4.2 contains 70 insiders in three scenarios (Table 3.1).

Table 3.1: The three insider scenarios of CERT release 4.2, following the answer key.

scenario	what the insider does	insiders
1	Starts working after hours and using a removable drive, which the user had not done before; uploads data to wikileaks.org and soon leaves.	30
2	Browses job websites and seeks work at a competitor; before leaving, copies data with a thumb drive at a much higher rate than before.	30
3	A disgruntled system administrator installs a keylogger on the supervisor’s computer, logs in as the supervisor with the captured credentials and sends an alarming mass email, then leaves.	10

The answer key also gives the time of each insider’s first and last malicious event. A user-day is labelled *malicious* if it falls within that window,

$$y(u, t) = \begin{cases} 1 & \text{if day } t \text{ lies between the first and last malicious event of insider } u, \\ 0 & \text{otherwise,} \end{cases} \quad (3.1)$$

and a user is malicious if any of their days is, $y(u) = \max_t y(u, t)$. This labels 1,883 user-days: 1,676 for scenario 2, whose insiders act over weeks, 187 for scenario 1 and 20 for scenario 3. A stricter label marks only the days on which a malicious event actually occurs; the feature extraction used in this thesis [6] records 966 such days (85, 861 and 20 by scenario). The language-model experiments use the window labels, and the PCA experiment uses the stricter labels. User-level results depend only on which users are malicious, so they are comparable across the two.

Malicious days are rare. At most 1,883 of the 330,295 user-days with computer activity (Section 3.4) are malicious, a fraction $\pi \leq 1,883/330,295 = 0.0057$. A rule that calls every day benign would therefore be correct on at least 99.4% of days while detecting nothing, so this thesis measures detection by ranking. A detector flags every observation whose score exceeds a threshold, and each threshold catches some fraction of the malicious cases (the true positive rate) at the cost of flagging some fraction of the benign ones (the false positive rate). The receiver operating characteristic (ROC) curve plots these two rates as the threshold moves from strict to lenient—a curve that rises steeply toward the top-left corner catches malicious cases before it flags many benign ones, while the diagonal is what random scores would give. The area under the curve (AUC) is the probability that a randomly chosen malicious case scores higher than a randomly chosen benign one, so 1 means perfect ranking and 0.5 means chance; Section 4.9 defines the user-level version used in this thesis. Malicious examples are also too few and too varied to learn from, so every detector is *one-class*: it is fit only on benign training data, $\mathcal{B} = \{D : y(D) = 0\}$ restricted to training users, and scores how far a new day departs from what it learned. The labels are used only for evaluation.

3.3 Data Preparation for PCA

Reconstruction Analysis

The method of Chapter II needs each observation as a vector of p numerical features, so the activity of each observation is summarized into one row $\mathbf{x}_i^\top$ of a matrix $X \in \mathbb{R}^{n \times p}$. Here an observation is a user-day, and its features are the day-level features of Le et al. [6]: counts of logons, removable-drive connections, file accesses, emails and web visits, each split by working hours and after hours and by the kind of computer, file or website involved (for example, visits to job-search, cloud-storage or leak websites), averages such as file and email sizes, whether the day is a weekday, and the user’s organizational and personality attributes. Release 4.2 gives $n = 330,452$ user-days with 502 features once identifiers and time indices are removed. Because PCA is sensitive to scale, each column is standardized with the mean and standard deviation of the training rows. Standardization needs a positive standard deviation, and 193 of the 502 columns never vary among the training days, so they are removed, leaving $p = 309$ features. Removing the organizational and personality attributes as well (ten of the 309; the eleventh, the business unit, takes a single value in this release) leaves 299, the PCA counterpart of the behavior-only score of Chapter IV.

PCA is fit on the 287,961 days of the 851 training users of Section 3.4, the users that also train the language model; all of them are benign, so PCA sees no insider and no held-out benign user. It keeps the first q principal directions U_q , with q the smallest number whose proportion of variance explained (2.5) reaches 95%; this gives $q = 71$ directions, which explain 95.1% of the variance ($q = 63$ without the profile attributes). Each observation is reduced to its scores and mapped back to the feature space with the inverse PCA transformation (2.6), and its anomaly score is its reconstruction error e_i (2.7); the mean squared error (2.8) summarizes a set of observations. Benign behavior is reconstructed well by the dominant directions of benign variation, so a large e_i marks an observation that those directions cannot explain.

The median reconstruction error is 6.6 on the days of the 79 held-out benign users and 17.0 on the 966 malicious days, and 6.4% of the malicious days exceed the 99th percentile of the training errors, against 0.5% of the held-out benign days. On the evaluation pool of Section 3.4 (60 insiders and 79 held-out benign users), scoring each user by their largest error gives a user-level AUC of 0.75 (95% interval [0.67, 0.83]), and 0.76 [0.67, 0.83] without the profile attributes; with all 70 insiders it is 0.78 [0.70, 0.85]. The threshold matters somewhat: keeping 80%, 90% or 99% of the variance ($q = 36, 55$ or 94) gives 0.81, 0.75 and 0.78. The removed columns matter as well. Among them are visits to leak and hacking websites and logons to the supervisor’s computer, which no training day contains and which occur, in the evaluation data, almost only on malicious days. Left in their raw units, as some software does for a column with zero standard deviation, they carry 99.7% of the malicious days’ squared error and raise the user-level AUC to 0.80 [0.73, 0.87]. On the same users, the language model’s full score reaches 0.65 and its behavior-only score 0.86 (Chapter IV).

3.4 Data Preparation for Language-Model Analysis

The language model reads activity grouped into *sessions*, each running from a logon to the matching logoff on one computer, with features following the extraction of Le et al. [6]: time of day, duration, computer, and counts of logon, device, file, email and web events. Release 4.2 has 470,611 sessions on 330,295 user-days; a user-day without a session has no text. Each user-day D is written as the text

$$T(D) = \text{DAY} \parallel \text{PSY} \parallel \text{SESSIONS} \parallel \text{SES}_1 \parallel \cdots \parallel \text{SES}_k,$$

lines joined by newlines: the organizational attributes (**DAY**), the personality scores (**PSY**), the number of sessions (**SESSIONS**) and one line of **key=value** features per session. Days have between one and nine sessions (two thirds have one), and the longest texts have about 1,300 tokens, well inside the model’s limit. Chapter IV shows a full example.

The model is Qwen3-8B [15], adapted with QLoRA [14]: the original weights are stored in 4 bits and kept fixed, and each adapted weight matrix receives a trainable correction of rank $r = 16$ (Section 4.5). Training minimizes the total surprisal of the benign training texts (4.6) in one pass over 287,827 benign user-days of 851 users.

The users are divided into three disjoint groups, $\mathcal{U} = \mathcal{U}_{\text{train}} \cup \mathcal{U}_{\text{val}} \cup \mathcal{U}_{\text{ins}}$. Every user with a labelled malicious day is an insider ($\mathcal{U}_{\text{ins}}$); each remaining user is held out ($\mathcal{U}_{\text{val}}$) with probability 0.1, by a fixed hash of the user’s identifier, and otherwise used for training (Table 3.2). The model never sees the insiders or the held-out benign users, so the two are compared on equal terms.

Table 3.2: User-disjoint splits of CERT release 4.2.

group	users	user-days	labelled malicious user-days
$\mathcal{U}_{\text{train}}$ (training)	851	287,827	0
$\mathcal{U}_{\text{val}}$ (held-out benign)	79	27,026	0
$\mathcal{U}_{\text{ins}}$ (insiders)	70	15,442	1,309

The results of Chapter IV and Chapter V use one evaluation pool: every day of the 60 insiders with at least one labelled malicious session-day and every day of the 79 held-out benign users, 40,519 user-days of 139 users of which 1,309 (3.2%) are labelled malicious. The 60 insiders are 29 from scenario 1, all 30 from scenario 2 and one from scenario 3, so scenario 2 supplies 89% of the malicious days. Each malicious user is ranked against the same 79 benign users.

[To complete: Label alignment (found 2026-09-28): in the data behind these counts, the labels were joined to the sessions on day numbers that count from two different start dates, so each malicious date’s label fell on the session-day five days earlier. Aligned, the pool would contain 70 insiders (30, 30 and 10 by scenario) and 1,355 malicious session-days. These numbers and the results of Chapter IV and Chapter V must be updated or explained once the correction is

made; see docs/LABEL_ALIGNMENT_2026-09-28.md in the repository.]

3.5 Experimental Design

The thesis runs three experiments on the same dataset (Table 3.3).

Table 3.3: The three experiments.

experiment	input	model	score of a day
PCA (Chapter II)	feature vector $\mathbf{x}$	PCA to q dimensions and its inverse	reconstruction error $\ \mathbf{x} - \hat{\mathbf{x}}\ _2^2$
Surprisal (Chapter IV)	text $T(D)$	Qwen3-8B with a rank-16 correction	average token surprisal $\bar{\ell}$, and its profile and behavior parts
Sparse features (Chapter V)	the model’s internal change at layer 26	sparse dictionary with 8,192 features, 4 active per token	change in $\bar{\ell}$ when selected features are edited

The language-model experiments follow five rules, and the PCA experiment follows the first four; the fifth does not apply, because PCA selects nothing with the labels.

1. **One-class training.** Models are fit on benign data only; labels are used for evaluation and, in Chapter V, for choosing which features to study.
2. **User-disjoint evaluation.** No user contributes days to both training and evaluation.
3. **A user score from day scores.** A user is scored by their most unusual day, $s(u) = \max_{D \in \mathcal{D}_u} s(D)$.
4. **Users as the unit of uncertainty.** Days of the same user are not independent, so intervals resample users rather than days.

5. **Selection and testing on different users.** Features chosen with labels are tested on malicious users not used in the choice; Chapter V uses 30 malicious users for selection and holds out the other 30.

Chapter IV

Language-Model Surprisal and Its Decomposition

4.1 Overview and Motivation

Chapter II scored an observation by how poorly a low-dimensional linear model reconstructs it. This chapter scores a user-day by how poorly a large language model, fine-tuned only on benign activity, predicts it. Both scores ask the same question: how badly does a model of normal behavior account for this observation? Two ideas from Chapter II carry over. The fine-tuning changes each weight matrix only by a low-rank correction (Section 4.5), and Chapter V decomposes the model’s internal change with a sparse relative of PCA. Scoring activity by how poorly a model trained on normal activity predicts it follows earlier work on log and insider-threat data [10, 11].

The new contribution of this chapter is to take the score apart. We show that it splits exactly into a part describing *who* the user is and a part describing *what* the user did, and that the first part actively hides malicious users.

4.2 Writing a User-Day as Text

Each *user-day*, one user on one calendar day, is serialized as text. The record below is a real benign day from CERT release 4.2 (user AAF0535, day 7). The session line is a single line in the actual text; it is wrapped here only to fit the page.

```
DAY week=1 project=na role=39 b_unit=0 f_unit=4 dept=15 team=11 itadmin=0
```

```

PSY 0=17 C=21 E=36 A=33 N=31

SESSIONS total=1 kept=1

SES idx=0 pc=0 isworkhour=1 isafterhour=0 isweekend=0 isweekendafterhour=0

duration=489 n_concurrent_sessions=1 start_with=1 end_with=1

ses_start=8.933 ses_end=17.083 n_allact=34 n_logon=1 n_usb=0

usb_mean_usb_dur=0 n_file=0 file_n-disk1=0 file_n_exef=0 n_email=3

email_mean_n_atts=1.333 n_http=29 http_n_jobf=0 http_n_cloudf=0

http_n_leakf=0 http_n_hackf=0

```

This user had one working-hours session on their usual computer, lasting 489 minutes, with 3 emails and 29 web requests. The lines fall into two classes. The **profile lines** (P), DAY and PSY, describe *who* the user is: organizational attributes and Big-Five personality scores, identical on every day of that user apart from the week number. The **behavior lines** (B), SESSIONS and SES, describe *what the user did* that day.

The model reads *tokens* $x_1, \dots, x_n$, short pieces of text in which numbers are split into single digits, so `dept=15` is the four tokens `dept`, `=`, `1`, `5`. A token's class is the class of the line it lies on,

$$c(x_t) = \begin{cases} P & \text{if } x_t \text{ lies on a DAY or PSY line,} \\ B & \text{if } x_t \text{ lies on a SESSIONS or SES line,} \end{cases}$$

so the class is decided by where a token sits, not by what it is: the digit 1 is a profile token in `team=11` and a behavior token in `n_logon=1`. The record above has $N = 212$ scored tokens, of which $N_P = 54$ are profile tokens and $N_B = 158$ behavior tokens; averaged over the malicious days of release 4.2, about 22% of scored tokens are profile tokens. Each token's surprisal is counted in the class of the token being predicted, while the model still reads the whole preceding text as context: the split decides which predictions are *counted*, not what the model *sees*.

4.3 Next-Token Probability and Surprisal

Before seeing token x_t , the model assigns a probability to every item of its vocabulary $\mathcal{V}$ (151,669 tokens). It first condenses the preceding tokens into a *context vector* $\mathbf{h}_{t-1} \in \mathbb{R}^d$, $d = 4096$, computed by a stack of 36 layers [12]. Each vocabulary item v has a learned vector $\mathbf{w}_v$, and its score, or *logit*, is the dot product $o_v = \mathbf{w}_v^\top \mathbf{h}_{t-1}$; all scores at once are $\mathbf{o} = W_{\text{out}} \mathbf{h}_{t-1}$. The *softmax* turns scores into probabilities,

$$p_\theta(x_t = v \mid x_{<t}) = \frac{e^{o_v}}{\sum_{v' \in \mathcal{V}} e^{o_{v'}}}, \quad (4.1)$$

where θ denotes the model’s parameters. Every probability is positive and they sum to 1; only differences between scores matter, since $p(a)/p(b) = e^{o_a - o_b}$; and no token is ever assigned probability zero.

The *surprisal* of the actual token is

$$\ell_t = -\log p_\theta(x_t \mid x_{<t}) = \text{LSE}(\mathbf{o}) - o_{x_t}, \quad \text{LSE}(\mathbf{o}) = \log \sum_v e^{o_v}, \quad (4.2)$$

where the log-sum-exp satisfies $o_{\max} \leq \text{LSE}(\mathbf{o}) \leq o_{\max} + \log |\mathcal{V}|$. Surprisal is therefore how far the actual token’s score falls below a soft maximum of all the scores: a token the benign-trained model scored near the top costs little, and a token it scored far below the others costs a lot.

4.4 Why the Negative Logarithm Measures Surprise

Two natural requirements on a measure of surprise $S(p)$ for an event of probability p leave only the logarithm, the measure of information introduced by Shannon [18]. First, rarer events are more surprising, so S decreases in p . Second, the surprises of independent events

add, and since $P(A \text{ and } B) = P(A)P(B)$ this means

$$S(pq) = S(p) + S(q) \quad \text{for all } p, q \in (0, 1]. \quad (4.3)$$

Writing $p = e^{-x}$ and $g(x) = S(e^{-x})$ turns (4.3) into Cauchy's equation $g(x+y) = g(x) + g(y)$, whose only increasing solutions are $g(x) = cx$. Hence

$$S(p) = -c \ln p, \quad c > 0, \quad (4.4)$$

where c only fixes the unit; this thesis uses $c = 1$, the *nat*. A certain event carries no surprise, an event thought impossible would be infinitely surprising, and every tenfold drop in probability adds the same $\ln 10$ nats.

For a single observed event, the surprisal is also its *negative log-likelihood*. By the chain rule of probability, the probability of a day's text is the product of each token's probability given the tokens before it, so

$$-\log p_\theta(x_2, \dots, x_n \mid x_1) = \sum_{t=2}^n -\log p_\theta(x_t \mid x_{<t}) = \sum_{t=2}^n \ell_t : \quad (4.5)$$

the total surprisal of a day is the negative log-likelihood of its text. Because days differ in length (about 200 to 530 tokens), the anomaly score of Section 4.6 divides this total by the number of scored tokens.

4.5 Learning From Benign Days Only

Fine-tuning chooses θ to minimize the total surprisal of the benign training text,

$$\theta^\star = \arg \min_{\theta} \sum_{D \in \mathcal{B}} \sum_t -\log p_\theta(x_t^{(D)} \mid x_{<t}^{(D)}), \quad (4.6)$$

where $\mathcal{B}$ is the set of benign training days and $x^{(D)}$ is the text of day D ; this is maximum-likelihood estimation. For the simplest model, one probability p_j for each of K tokens, minimizing $\sum_j N_j(-\log p_j)$ subject to $\sum_j p_j = 1$ with a Lagrange multiplier gives

$$\hat{p}_j = \frac{N_j}{N}, \quad (4.7)$$

the benign frequencies: values common among benign users become cheap and rare values expensive, which is why surprisal under a benign-trained model works as an anomaly score. Fine-tuning performs the same minimization for a model whose probabilities depend on the whole preceding text, by gradient descent.

Fine-tuning does not change every weight. It adds a low-rank correction and keeps the original weights frozen [13]: $W' = W + GH$ with $G \in \mathbb{R}^{d \times r}$, $H \in \mathbb{R}^{r \times d}$ and $r = 16$. For a 4096×4096 matrix the correction has 131,072 entries, 0.78% of the 16,777,216 in the matrix, and whatever the input $\mathbf{x}$, the change $GH\mathbf{x}$ lies in the span of the 16 columns of G . Just as the rank- q reconstruction of Chapter II keeps only q directions, fine-tuning can change each adapted matrix's output along at most 16 directions; the parallel is structural only.

4.6 The Anomaly Score of a Day and of a User

Let $\mathcal{T}$ be the set of scored positions of a day (every token except the first, which has nothing before it) and $N = |\mathcal{T}|$. The anomaly score of the day is its average token surprisal,

$$\bar{\ell} = \frac{1}{N} \sum_{t \in \mathcal{T}} \ell_t. \quad (4.8)$$

A high $\bar{\ell}$ means that a model trained only on benign days found this day hard to predict. A user's score is the score of their most unusual day,

$$\bar{\ell}(u) = \max_{D \in \mathcal{D}_u} \bar{\ell}(D),$$

where $\mathcal{D}_u$ is the set of that user's days.

4.7 Splitting the Score by Token Class

Divide the scored positions into profile and behavior positions, $\mathcal{T} = \mathcal{T}_P \cup \mathcal{T}_B$, with $N = N_P + N_B$, class totals L_P and L_B , and class means $\bar{\ell}_P = L_P/N_P$ and $\bar{\ell}_B = L_B/N_B$. Then

$$\bar{\ell} = \frac{1}{N}(L_P + L_B) = \frac{N_P}{N} \bar{\ell}_P + \frac{N_B}{N} \bar{\ell}_B : \quad (4.9)$$

the day's score is exactly a weighted average of the profile mean and the behavior mean, weighted by their shares of the tokens. The behavior-only score

$$\bar{\ell}_B = \frac{L_B}{N_B} = \frac{L - L_P}{N - N_P} \quad (4.10)$$

uses the same model with no retraining; it simply stops counting profile tokens. Subtracting the profile mean instead does not work, because

$$\bar{\ell} - \bar{\ell}_P = \frac{N_B}{N} (\bar{\ell}_B - \bar{\ell}_P) \quad (4.11)$$

measures how much harder the behavior was to predict than the profile, not how hard the behavior was to predict, and it can reverse the order of two days.

4.8 A Numerical Example

Two user-days each have five scored tokens, one profile token and four behavior tokens, so $N_P/N = 0.2$. Day A belongs to a malicious user with a common profile and unusual behavior; Day B to a benign user with a rare profile and routine behavior.

Table 4.1: Class means and scores of the two example days.

day	$\bar{\ell}_P$	$\bar{\ell}_B$	full score $\bar{\ell} = 0.2 \bar{\ell}_P + 0.8 \bar{\ell}_B$	behavior-only score
A (malicious)	0.1054	0.9324	0.7670	0.9324
B (benign)	2.9957	0.4125	0.9292	0.4125

The full score ranks the benign Day B above the malicious Day A: its profile part favors Day B by 0.578 and outweighs a behavior part that favors Day A by 0.416. The behavior-only score restores the correct order. The naive subtraction can fail as well: for a Day C with $\bar{\ell}_P = 3.0$, $\bar{\ell}_B = 1.2$ and a Day D with $\bar{\ell}_P = 0.2$, $\bar{\ell}_B = 0.8$, equation (4.11) gives $0.8(1.2 - 3.0) = -1.44$ and $0.8(0.8 - 0.2) = 0.48$, ranking D above C although C’s behavior is less predictable.

4.9 Measuring Detection: The User-Level AUC

With one malicious user m compared against n_- benign users $u_1, \dots, u_{n_-}$, the area under the ROC curve for m is the fraction of benign users that m outranks,

$$\text{AUC}_m = \frac{1}{n_-} \sum_{j=1}^{n_-} \mathbb{I}[\bar{\ell}(m) > \bar{\ell}(u_j)], \quad (4.12)$$

where $\mathbb{I}[\cdot]$ is 1 when the condition holds and 0 otherwise (a tie counts as $\frac{1}{2}$). Averaging over M malicious users gives $\text{AUC} = \frac{1}{M} \sum_m \text{AUC}_m$. Because every malicious user is compared with the same benign users, this average equals the pooled AUC over all $M \times n_-$ malicious–benign pairs:

$$\frac{1}{M} \sum_m \frac{1}{n_-} \sum_j \mathbb{I}[\cdot] = \frac{1}{M n_-} \sum_m \sum_j \mathbb{I}[\cdot].$$

A value of 0.5 is chance, 1 is perfect, and a value below 0.5 means malicious users are systematically ranked *below* benign ones.

For example, suppose four benign users have full scores (0.93, 0.70, 0.85, 0.60) and a

malicious user has full score 0.77. The malicious user outranks 0.70 and 0.60, so $AUC_m = 2/4 = 0.50$. If the behavior-only scores are (0.41, 0.55, 0.70, 0.48) for the benign users and 0.93 for the malicious user, it outranks all four, and $AUC_m = 4/4 = 1.00$.

4.10 Core Finding

Table 4.2 scores the same 60 malicious and 79 benign users of CERT release 4.2 with four versions of the score, all computed from one set of model outputs.

Table 4.2: User-level AUC on CERT release 4.2 (60 malicious users, 79 benign users, none seen in training) for scores built from different token classes. Intervals are 95% bootstrap intervals, resampling malicious users.

score	tokens counted	user AUC	95% interval
full score $\bar{\ell}$	all	0.65	[0.58, 0.72]
behavior only $\bar{\ell}_B$	behavior	0.86	[0.83, 0.89]
profile only $\bar{\ell}_P$	profile	0.47	[0.40, 0.54]
organization line only	DAY	0.22	[0.15, 0.29]

Scoring behavior tokens alone raises the user-level AUC by +0.21, with a 95% interval of [+0.16, +0.27]. Compared one malicious user at a time, 51 of the 60 malicious users are ranked higher among the benign users, 3 are unchanged, and 6 are ranked lower.

Three observations explain the result.

- **The profile tokens are not neutral noise.** On their own they rank malicious users below benign ones (0.47), and the organization line alone does so strongly (0.22). In this benchmark, malicious users tend to have common profiles, so their profile tokens are cheap and pull their full scores down: exactly Day A of the worked example.
- **Surprisal measures typicality, not suspicion.** Every evaluated user is new to the

model, so a profile’s surprisal reflects how common that kind of employee was among benign training users. A rare but innocent job title is surprising; a malicious user with an ordinary job title is not.

- **The fix requires no new model.** The decomposition (4.9) is exact, so the behavior-only score is obtained from the same model outputs by not counting profile tokens. It removes the profile’s direct contribution to the score, not its influence: the profile is still in the context, so every behavior-token prediction is still conditioned on it.

The same check applies to any detector whose input mixes who a user is with what they did.

The user-level AUC measures how well users are ranked. It does not show that the score pinpoints the specific malicious day, and it should not be read that way.

Chapter V

Interpretable Dimensionality Reduction of the Fine-Tuned Model

5.1 Overview and Motivation

Chapter IV examined the score from the outside: which tokens its surprisal comes from. *Mechanistic interpretability* asks what the model computes internally [16, 17]. This chapter answers that question with a form of dimensionality reduction designed for interpretation. PCA (Chapter II) reduces data to a few dense directions, each of which mixes many original features. A *sparse autoencoder* instead describes each observation with a few directions drawn from a much larger dictionary; because only a few are active at once, each direction, or *feature*, tends to mean one recognizable thing. We apply it to the change that benign fine-tuning made to the model’s internal representations, and test two things about the features that best separate malicious from benign days: where they activate, and whether editing them changes the anomaly score.

5.2 Hidden States and the Adaptation Difference

The model processes text through a stack of 36 layers. After each layer, each token t is represented by a *hidden state* $\mathbf{h}_t \in \mathbb{R}^d$ with $d = 4096$. Running the same text through the model before fine-tuning (the *base* model) and after it (the *adapted* model) gives two hidden

states for every token. Their difference,

$$\boldsymbol{\delta}_t = \mathbf{h}_t^{\text{ada}} - \mathbf{h}_t^{\text{base}} \in \mathbb{R}^d, \quad (5.1)$$

is exactly what benign fine-tuning added to the representation of token t . We study the hidden states after layer 26.

5.3 A Sparse Dictionary for the Difference

Recall the PCA reconstruction of Chapter II: an observation is encoded by $\mathbf{z} = U_q^\top \mathbf{x}$ using $q < p$ orthonormal directions, decoded by $\hat{\mathbf{x}} = U_q \mathbf{z}$, and the directions minimize the reconstruction error (2.7). A *sparse autoencoder* (SAE) keeps the reconstruction objective but changes two things: it uses *more* directions than dimensions ($m > d$), and it lets only a few of them be active for any one token [19].

To encode, compute a score for each of the m dictionary directions and keep only the k largest:

$$\mathbf{a}_t = W_{\text{enc}}(\boldsymbol{\delta}_t - \mathbf{b}) \in \mathbb{R}^m, \quad \mathbf{z}_t = \text{TopK}_k(\mathbf{a}_t), \quad (5.2)$$

where TopK_k keeps the k largest entries and sets the rest to zero.

The difference is then decoded from the active directions:

$$\hat{\boldsymbol{\delta}}_t = W_{\text{dec}} \mathbf{z}_t + \mathbf{b} = \sum_{f: z_{f,t} \neq 0} z_{f,t} \mathbf{w}_f + \mathbf{b}, \quad (5.3)$$

where $\mathbf{w}_f$ is column f of W_{dec} , the direction of *feature* f .

Training chooses W_{enc} , W_{dec} and $\mathbf{b}$ to minimize $\sum_t \|\boldsymbol{\delta}_t - \hat{\boldsymbol{\delta}}_t\|_2^2$, the same squared reconstruction error as PCA. No labels are used. (Each coordinate of $\boldsymbol{\delta}$ is standardized before encoding and restored after decoding; we suppress this in the notation.)

Table 5.1: PCA and the sparse autoencoder side by side.

	PCA (Chapter II)	sparse autoencoder
number of directions	$q < p$	$m > d$
directions	orthonormal	not orthogonal
active per observation	all q	only k
objective	$\min \ \mathbf{x} - \hat{\mathbf{x}}\ ^2$	$\min \ \boldsymbol{\delta} - \hat{\boldsymbol{\delta}}\ ^2$
our setting	—	$d = 4096, m = 8192, k = 4$

With $m = 8,192$ directions but only $k = 4$ active per token, each token’s $\boldsymbol{\delta}_t$ is explained by four features. Because so few are active at once, each feature tends to specialize in one recognizable pattern. Our dictionary is trained on 2,000,000 token-level differences.

5.4 A Numerical Example of Sparse Coding

Take $d = 2$, a dictionary of $m = 3$ unit directions $\mathbf{w}_1 = (1, 0)$, $\mathbf{w}_2 = (0, 1)$ and $\mathbf{w}_3 = (0.6, 0.8)$, one active feature ($k = 1$), $\mathbf{b} = \mathbf{0}$ and $W_{\text{enc}} = W_{\text{dec}}^\top$. For $\boldsymbol{\delta} = (1.2, 1.6)$ the scores are $\mathbf{a} = (1.2, 1.6, 2.0)$, so feature 3 is kept and $\hat{\boldsymbol{\delta}} = 2.0 \mathbf{w}_3 = (1.2, 1.6)$ with zero error. With only the two axes available, the best single direction would leave an error of $1.2^2 = 1.44$. A different token, $\boldsymbol{\delta}' = (1.0, 0.1)$, is described by feature 1 with error 0.01: each token uses the few features that fit it best, which is *sparse coding*.

5.5 Choosing the Features to Test

For each feature f , its *gap* compares how strongly it fires on malicious and benign days,

$$g_f = \bar{z}_f^{(+)} - \bar{z}_f^{(-)}, \quad (5.4)$$

where $\bar{z}_f^{(+)}$ and $\bar{z}_f^{(-)}$ are the average activations of feature f over the tokens of malicious days and of benign days. The *selected set* $\mathcal{F}$ is the five features with the largest gaps. The *control set* $\mathcal{C}$ is the five features with the gaps closest to zero among those active on at least 0.2% of tokens. The threshold keeps rarely firing features out of the control set, but it does not match the two sets’ activity: in the held-out test below, the control features were active on 0.36% of tokens on average and the selected features on 0.18%. The control answers the question: does editing *any* active features change the score, or only the selected ones?

5.6 Where Features Fire: Token-Class Attribution

For a feature f and a token class c , the share of the feature’s total activation that lands on class- c tokens is

$$\rho_f(c) = \frac{\sum_{t \in \mathcal{T}_c} z_{f,t}}{\sum_{t \in \mathcal{T}} z_{f,t}}, \quad (5.5)$$

and its *enrichment* compares that share with the class’s share of the tokens,

$$E_f(c) = \frac{\rho_f(c)}{N_c/N}.$$

An enrichment of 1 means the feature fires on class c exactly as often as chance would suggest; larger values mean it prefers class c , up to the maximum $1/(N_c/N)$ reached when all of a feature’s activation lands on that class.

5.7 Causal Test: Editing Features and Re-Scoring

Attribution shows where a feature fires, not whether it matters. The causal test edits the features and measures the change in the anomaly score. Take a malicious *receiver* day. A *donor* day comes from another user in the same department, and it is one of two kinds: a benign day of a colleague, or a malicious day of another malicious user. Comparing the two kinds of donor asks whether the benign colleague’s values in particular make the receiver

look normal, rather than any edit of the same features.

The test has six steps.

1. Encode each receiver token with (5.2).
2. On every token where at least one selected feature is active, replace the activation of *every* selected feature, including features that were zero at that token, by $z'_{f,t} = (1 - \gamma) z_{f,t} + \gamma \bar{z}_f^{\text{donor}}$ for $f \in \mathcal{F}$, with patch strength $\gamma \in \{0.25, 0.5, 0.75, 1\}$; all other features are unchanged. Here $\bar{z}_f^{\text{donor}}$ is the average activation of feature f over the donor day's tokens on which at least one selected feature is active.
3. Decode the edited code, $\hat{\boldsymbol{\delta}}'_t = W_{\text{dec}} \mathbf{z}'_t + \mathbf{b}$, and replace the adapted hidden state at layer 26 by $\mathbf{h}'_t = \mathbf{h}_t^{\text{ada}} + (\hat{\boldsymbol{\delta}}'_t - \boldsymbol{\delta}_t) = \mathbf{h}_t^{\text{base}} + \hat{\boldsymbol{\delta}}'_t$. If any token of the day has an active selected feature, this replacement is made at every token of the day: tokens without one receive their unedited reconstruction, and a day with no active selected feature is left unchanged. The reconstruction is not exact, so it can move the score even without an edit; step 6 is built so that this movement cancels.
4. Run the remaining layers from $\mathbf{h}'$ and recompute the day's score $\bar{\ell}'$; the change $\delta = \bar{\ell}' - \bar{\ell}$ is negative when the edit makes the day look less anomalous.
5. For each receiver day, try up to 16 candidate donor days of each kind at all four patch strengths, and keep the most negative change for each kind, giving $\delta_{\mathcal{F},\text{ben}}$ and $\delta_{\mathcal{F},\text{mal}}$; the control set gives $\delta_{\mathcal{C},\text{ben}}$ and $\delta_{\mathcal{C},\text{mal}}$.
6. Compare donors, then feature sets, through the donor advantages $a_{\mathcal{F}} = \delta_{\mathcal{F},\text{mal}} - \delta_{\mathcal{F},\text{ben}}$ and $a_{\mathcal{C}} = \delta_{\mathcal{C},\text{mal}} - \delta_{\mathcal{C},\text{ben}}$.

The effect of the selected features is the average, over receiver days that have all four values, of

$$\tau = a_{\mathcal{F}} - a_{\mathcal{C}} = (\delta_{\mathcal{F},\text{mal}} - \delta_{\mathcal{F},\text{ben}}) - (\delta_{\mathcal{C},\text{mal}} - \delta_{\mathcal{C},\text{ben}}). \quad (5.6)$$

The unedited score cancels, and so, within one feature set, does the part of the reconstruction’s effect that simply adds to the edit’s; the layers after the edit are nonlinear, so the rest does not cancel. A positive τ means that moving the selected features toward a benign colleague, rather than toward another malicious user, lowers the score by more than the same comparison does for the control features. A simpler comparison, $\delta_{\mathcal{C},\text{ben}} - \delta_{\mathcal{F},\text{ben}}$, uses benign donors only. It is a different quantity: it mixes the effect of moving toward benign values with any effect of editing the selected features at all, which τ removes, and the interval reported below belongs to τ only.

5.8 Core Mechanistic Finding

Table 5.2 gives the attribution (5.5) of the five features selected on all 60 malicious users of CERT release 4.2.

Table 5.2: Share of each selected feature’s activation on each token class, over malicious days of CERT release 4.2 (layer 26). Session (SES) lines are 74.9% of these tokens, the organization (DAY) line 12.6%, and the personality (PSY) line 9.0%.

feature	$\rho_f(\text{SES})$	$\rho_f(\text{DAY})$	$\rho_f(\text{PSY})$	enrichment
4596	0.999	0.000	0.001	1.334 on SES
3673	1.000	0.000	0.000	1.335 on SES
3455	1.000	0.000	0.000	1.335 on SES
1268	1.000	0.000	0.000	1.335 on SES
2302	0.001	0.998	0.001	7.907 on DAY

Four of the five features put at least 99.9% of their activation on session tokens. Their enrichment of 1.334 to 1.335 is the maximum possible, since $1/0.7492 = 1.335$: essentially all of their activation lands on the session lines. The strongest activations of one of them (3673) all fall on the `_dur` token that names the session-duration fields. The fifth feature

(2302) fires on the organization line instead.

This table answers one question only: *where* each feature activates. It does not say what information a feature represents; a feature that activates on session tokens could still carry information about who the user is, such as the session lengths typical of a department. Nor does it say which predictions change when the feature is edited. Those are separate questions.

For the causal test, the 60 malicious users were split in half. Features were re-selected using only the 30 *discovery* users; the re-selected set shares four of the five features above. The test was then run on the other 30 malicious users, whom feature selection never saw. With donors from the same department, (5.6) gave

$$\tau = 0.00076 \text{ nats}, \quad 95\% \text{ interval } [0.00035, 0.00121],$$

over the 605 held-out malicious days with all four values, and τ was positive for 21 of 29 held-out users. The interval resamples users. In the same data the donor advantage was $a_{\mathcal{F}} = 0.00063$ for the selected features and $a_{\mathcal{C}} = -0.00012$ for the controls. The benign-only comparison is 0.0028 here; it has no interval of its own. The estimate is positive in all four organizational groupings tested (department, department and role, team, role: 0.00034 to 0.00076 nats); the department grouping is the one whose interval excludes zero.

A later rescoring of the same 638 held-out malicious days repeated every one of these edits exactly and recorded the change in loss separately for profile tokens and behavior tokens. It reproduces the value above to six decimals when step 5 keeps the best candidate. When the candidates are averaged instead of the best one being kept, at full strength ($\gamma = 1$), the same comparison is negative in every grouping: -0.0028 nats for department on the average over users, with 95% interval $[-0.0045, -0.0012]$, and intervals that include zero for the other three. Averaging over all four strengths also gives negative values. The positive τ above therefore describes the best of the searched edits. It does not show that a typical edit

toward a benign colleague makes a malicious day look normal.

Split by token class, with benign donors at full strength and averaged over donors, the selected edits raise the loss on behavior tokens by 0.016 to 0.018 nats per token more than the control edits do, in every grouping, with every interval excluding zero. At strengths 0.25 and 0.5 the difference is slightly negative. The difference on profile tokens is uncertain (every interval includes zero), and the direct comparison between the two classes, which was declared as the main quantity before the rescoring, is inconclusive. So the result shows neither that the edits leave profile predictions alone nor that the two effects differ. The profile lines come before the session lines, and each token is predicted only from earlier tokens, so an edit on a session token can change behavior predictions but not profile predictions. The edit is joint: at the organization-line tokens where feature 2302 fires, every selected feature is written, so any profile effect cannot be credited to feature 2302 alone.

Three things are established. The features that most distinguish malicious days activate almost only on session tokens. On users held out from feature selection, the best of the searched edits favors a benign donor more for the selected features than for the controls, by a small margin, while the average edit does not. And the joint edit of the selected features raises behavior-token loss more than the control edit at the two larger strengths. Several things are not established: what information the features represent; whether the behavior effect exceeds what an equally large change of the hidden state at the same tokens would cause, since the selected edits are several times larger than the control edits; whether it is specific to malicious days, since every edited day was malicious; whether the edits change profile predictions; and whether the profile shortcut of Chapter IV arises mainly from averaging surprisal over every token, since the score decomposition shows that the shortcut is removable at scoring time but not where it arises inside the model.

The best-candidate effect is small in absolute terms (0.00076 nats against day scores near 1), and it is about half the size of the estimate obtained before the held-out split.

5.9 Limitations

- The dictionary’s layer, size and sparsity (layer 26, $m = 8,192$, $k = 4$) were chosen among twelve candidate settings using runs that included all 60 malicious users. The features were re-selected on held-out users, but this setting was not.
- All results come from a single fine-tuned model, and their stability across training seeds is untested. A companion-dataset result once cited here as a seed reversal selected its features from an undefined ranking and has been withdrawn.
- Step 5 of the test keeps the best of many candidates. That favours whichever set’s candidate scores vary more, and there can be fewer malicious than benign donors, so τ can be positive without a donor-specific effect. Averaged over all candidates, the comparison is negative.
- The controls pass an activity threshold but are not activity-matched to the selected features, and the selected edits change the hidden state several times more than the control edits.
- Every edit replaces the hidden-state change with the dictionary’s reconstruction at every token of the day before editing, and edits all selected features together. A version that keeps the original hidden state and adds only the edit, $\mathbf{h}' = \mathbf{h} + W_{\text{dec}}(\mathbf{z}' - \mathbf{z})$, has not been used for the reported results.
- One benchmark release was used, CERT release 4.2. It is synthetic, and it was evaluated many times during development.
- The user-level AUC ranks users; it does not show that the model identifies the specific malicious day.

REFERENCES

Bibliography

- [1] Pearson, K. (1901). On lines and planes of closest fit to systems of points in space. *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, 2(11), 559–572.
- [2] Hotelling, H. (1933). Analysis of a complex of statistical variables into principal components. *Journal of Educational Psychology*, 24(6), 417–441.
- [3] Jolliffe, I. T. (1986). *Principal Component Analysis*. Springer Series in Statistics. Springer, New York. <https://doi.org/10.1007/978-1-4757-1904-8>
- [4] Software Engineering Institute. (2016). *Insider Threat Test Dataset*. Carnegie Mellon University Software Engineering Institute. <https://doi.org/10.1184/R1/12841247.v1>
- [5] Glasser, J., & Lindauer, B. (2013). Bridging the gap: A pragmatic approach to generating insider threat data. In *2013 IEEE Security and Privacy Workshops* (pp. 98–104). IEEE. <https://doi.org/10.1109/SPW.2013.37>
- [6] Le, D. C., Zincir-Heywood, N., & Heywood, M. I. (2020). Analyzing data granularity levels for insider threat detection using machine learning. *IEEE Transactions on Network and Service Management*, 17(1), 30–44. <https://doi.org/10.1109/TNSM.2020.2967721>
- [7] Shlens, J. (2014). *A Tutorial on Principal Component Analysis*. arXiv. <https://doi.org/10.48550/arXiv.1404.1100>

- [8] Cappelli, D., Moore, A., & Trzeciak, R. (2012). *The CERT Guide to Insider Threats: How to Prevent, Detect, and Respond to Information Technology Crimes (Theft, Sabotage, Fraud)*. Addison-Wesley Professional.
- [9] HPC Wiki, The University of Southern Mississippi. (2024). *Main Page*. Magnolia HPC Wiki. Suggested infrastructure acknowledgment: The authors acknowledge HPC at The University of Southern Mississippi supported by the National Science Foundation under the Major Research Instrumentation (MRI) program via Grant # ACI 1626217. http://magnolia.usm.edu/index.php?title=Main_Page&oldid=1116
- [10] Du, M., Li, F., Zheng, G., & Srikumar, V. (2017). DeepLog: Anomaly detection and diagnosis from system logs through deep learning. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security*.
- [11] Tuor, A., Kaplan, S., Hutchinson, B., Nichols, N., & Robinson, S. (2017). Deep learning for unsupervised insider threat detection in structured cybersecurity data streams. In *AAAI Workshop on Artificial Intelligence for Cyber Security*.
- [12] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems*.
- [13] Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2022). LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*.
- [14] Dettmers, T., Pagnoni, A., Holtzman, A., & Zettlemoyer, L. (2023). QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems*.
- [15] Qwen Team. (2025). *Qwen3 technical report*. arXiv preprint arXiv:2505.09388.

- [16] Bricken, T., Templeton, A., Batson, J., Chen, B., Jermyn, A., Conerly, T., Turner, N. L., et al. (2023). Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*. <https://transformer-circuits.pub/2023/monosemantic-features>
- [17] Cunningham, H., Ewart, A., Riggs, L., Huben, R., & Sharkey, L. (2023). Sparse autoencoders find highly interpretable features in language models. arXiv preprint arXiv:2309.08600.
- [18] Shannon, C. E. (1948). A mathematical theory of communication. *The Bell System Technical Journal*, 27(3), 379–423.
- [19] Gao, L., Dupré la Tour, T., Tillman, H., Goh, G., Troll, R., Radford, A., Sutskever, I., Leike, J., & Wu, J. (2024). Scaling and evaluating sparse autoencoders. arXiv preprint arXiv:2406.04093.